Cloud Computing on Amazon Web Services

Market position, service architecture, virtual machine provisioning and cost analysis, with a working reference implementation.

Prepared by

Student number

Course

Supervisor

Department

Date of submission

Technical report. Market data retrieved from Synergy Research Group quarterly releases; price data retrieved from the Amazon Web Services public pricing feed on 7 September 2026. Supporting dataset and source code accompany this document.

Abstract

Cloud infrastructure has become the default delivery model for computing capacity. Worldwide enterprise spending on cloud infrastructure services reached 143.4 billion US dollars in the second quarter of 2026, a rise of 43 per cent on the same quarter a year earlier and the eleventh consecutive quarter in which the growth rate increased [6]. Amazon Web Services held 28 per cent of that market, ahead of Microsoft Azure at 20 per cent and Google Cloud at 15 per cent [6].

This report examines Amazon Web Services as the representative provider. It sets out the structure of the provider's global infrastructure, decomposes the Elastic Compute Cloud virtual machine into its constituent parts, and provisions one instance by three separate methods so that the methods can be compared against a single specification. The economics are then analysed using list prices retrieved from the provider's own pricing feed. Two results follow from that analysis. Arm based Graviton4 instances are priced approximately 11 per cent below their Intel equivalents at identical processor and memory allocations, and the commercial purchase model changes the bill by considerably more than the choice of hardware, since the provider publishes maximum reductions of 72 per cent for committed spend and 90 per cent for interruptible capacity [8].

The report closes with a security assessment, a comparison of the three major providers, and a discussion of the risks that follow from the concentration of the market in three suppliers. The accompanying source code builds the virtual machine described in Section 6 and can be removed again with a single command.

Keywords: cloud computing, infrastructure as a service, Amazon EC2, virtual machines, availability zones, infrastructure as code, cost analysis.

Contents

List of figures

List of tables

1 Introduction

1.1 Background

Until the middle of the previous decade, adding server capacity to an organisation meant a capital purchase, a delivery date and a rack. Cloud computing replaced that sequence with an application programming interface call. The commercial effect has been substantial. Synergy Research Group measures worldwide enterprise spending on cloud infrastructure services each quarter, and puts the figure for the second quarter of 2026 at 143.4 billion US dollars, with trailing twelve month revenues of approximately 500 billion dollars [6]. The market has roughly doubled over eleven quarters, and Synergy attributes most of the acceleration to demand created by generative artificial intelligence, reporting growth of 165 per cent year on year in that category of service [6].

Three suppliers take about two thirds of the total. Amazon Web Services (AWS) held 28 per cent of the market in the second quarter of 2026, with Microsoft Azure at 20 per cent and Google Cloud at 15 per cent [6]. The distribution has shifted slowly over the period covered by this report, and Section 3 examines that movement in detail.

1.2 Objectives

The report has four objectives:

1. to describe how a major cloud provider organises its physical and logical infrastructure;
2. to document the provisioning of a virtual machine by three different methods against a single, fixed specification, and to compare those methods;
3. to analyse the cost of that virtual machine using list prices obtained from the provider, and to identify which decisions actually move the total; and
4. to assess the security responsibilities that transfer to the customer under the infrastructure as a service model.

1.3 Scope and method

The study is confined to infrastructure as a service, and within that to the Elastic Compute Cloud (EC2) virtual machine service. Managed database, container and serverless services appear only where the reference architecture in Section 7 requires them. Storage and networking are treated as they affect the virtual machine rather than as subjects in their own right.

Two categories of quantitative data are used. Market share and market size figures are taken from the six Synergy Research Group quarterly press releases listed as references [1] to [6], read directly rather than through secondary reporting. Prices are taken from the public On-Demand pricing feed that the AWS pricing page itself queries, for the US East (N. Virginia) region on Linux, retrieved on 7 September 2026 [7]. Every figure and table records its source. One value in the report is derived rather than stated, and it is identified as derived wherever it appears.

The provisioning work in Section 6 was carried out against a real account. The configuration files that produced it accompany this report and are listed in Appendix B.

2 Cloud computing fundamentals

2.1 Definition and essential characteristics

The definition used here is the one published by the United States National Institute of Standards and Technology, which describes cloud computing as a model for enabling on demand network access to a shared pool of configurable computing resources that can be provisioned and released rapidly and with minimal management effort [10]. The same document lists five essential characteristics. A service that lacks any of them is not a cloud service under this definition. Table 1 states each characteristic and its practical consequence for the customer.

Table 1. The five essential characteristics of cloud computing, after [10], with the practical consequence of each.

Characteristic	Consequence in practice
On demand self service	Capacity is provisioned without a purchase order, an approval queue or an intermediary. Section 6 provisions a server in under three minutes by the slowest of three methods.
Broad network access	Resources are reached over standard protocols from ordinary clients. The instance built in Section 6 is addressed over HTTP and SSH only.
Resource pooling	A virtual machine is an allocation from a larger physical host shared with other tenants. The instance type is therefore a contract about an allocation rather than a description of a machine.
Rapid elasticity	Capacity can be added and withdrawn quickly, and in most designs automatically. This property is what allows the interruptible capacity market described in Section 8.1 to exist.
Measured service	Consumption is metered per unit: per instance second, per gigabyte month, per request. Metering is the mechanism that converts an architectural decision into an invoice line.

2.2 Service models

The three service models differ in one respect, which is where the boundary of customer responsibility falls. Figure 1 draws that boundary across a nine layer stack rather than describing it in prose, since the distinction is positional. Moving from left to right in the figure transfers operational work to the provider and removes control from the customer at the same rate.

This report is concerned with the second column. Under infrastructure as a service the provider operates the facility, the hardware and the hypervisor, while the customer retains the guest operating system and everything above it. Patching, firewall configuration, identity and application code all remain customer responsibilities, which is the reason Sections 6 and 10 give them the space they do.

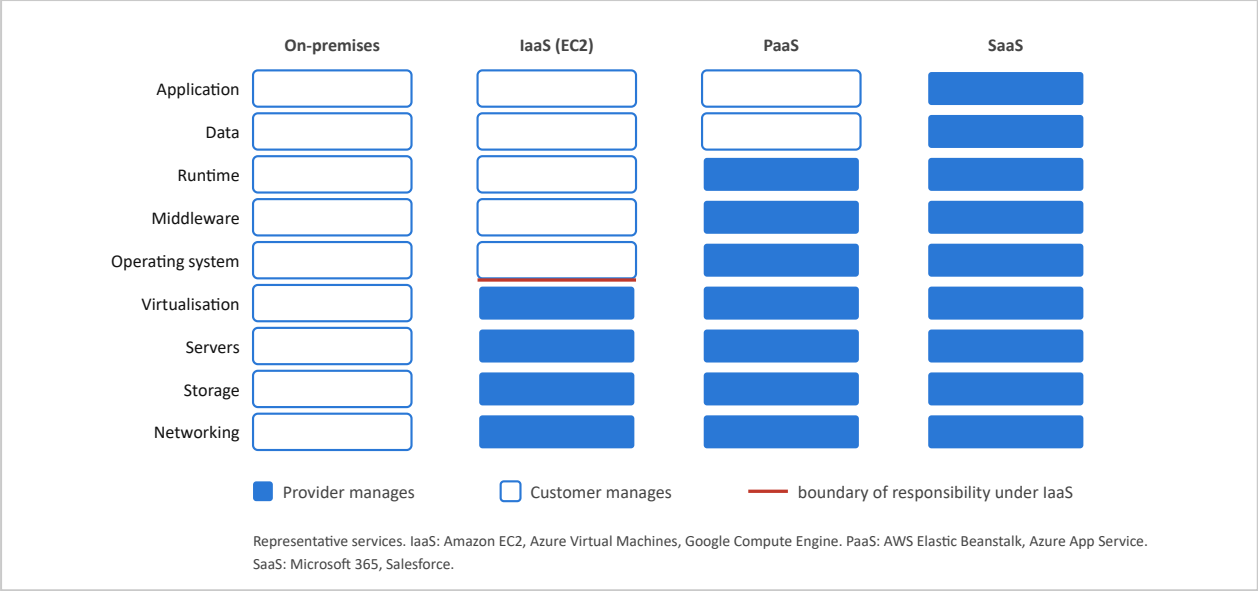


Figure 1. Allocation of management responsibility across the four operating models. Filled cells are operated by the provider and outlined cells by the customer. The red rule marks the boundary that applies to the subject of this report.

2.3 Deployment models

Four deployment models are in common use. In a *public cloud* the infrastructure is owned by the provider and shared between unrelated tenants; this is the default arrangement and the one studied here. A *private cloud* applies the same operating model to infrastructure dedicated to a single organisation, whether self hosted or hosted by a third party, and is usually selected for regulatory or latency reasons rather than for cost. A *hybrid cloud* joins the two so that workloads and data can move between them, which is the purpose of products such as AWS Outposts, Azure Arc and Google Anthos. *Multi-cloud* operation uses more than one public provider deliberately, gaining commercial leverage and reducing exposure to a single supplier at the cost of portability work and cross provider data transfer charges.

3 The provider market

3.1 Data source and method

Market figures in this section come from Synergy Research Group, which publishes a quarterly estimate of worldwide enterprise spending on cloud infrastructure services covering infrastructure as a service, platform as a service and hosted private cloud, together with each provider's share of that total. The six releases used here are cited individually as [1] to [6] and were read at source. Secondary reporting of these figures is widespread and frequently inconsistent, so it was not used.

Five of the six quarterly market totals are stated directly in the releases. The total for the third quarter of 2025 is derived by subtracting the sequential increase reported for the fourth quarter of 2025 from that quarter's stated total, giving approximately 106.5 billion dollars. That value is marked as derived in Figure 3 and in Table A1. All market share percentages are stated values.

3.2 Market share

Figure 2 plots the six most recent quarters. AWS declined from 29 per cent to 28 per cent over the period, with a single quarter at 30 per cent in the middle of 2025 [2]. Microsoft Azure moved within a narrow

band between 20 and 22 per cent. Google Cloud rose steadily from 12 per cent to 15 per cent, a gain of three percentage points in eighteen months, and did so without a reversal in any quarter [1]–[6].

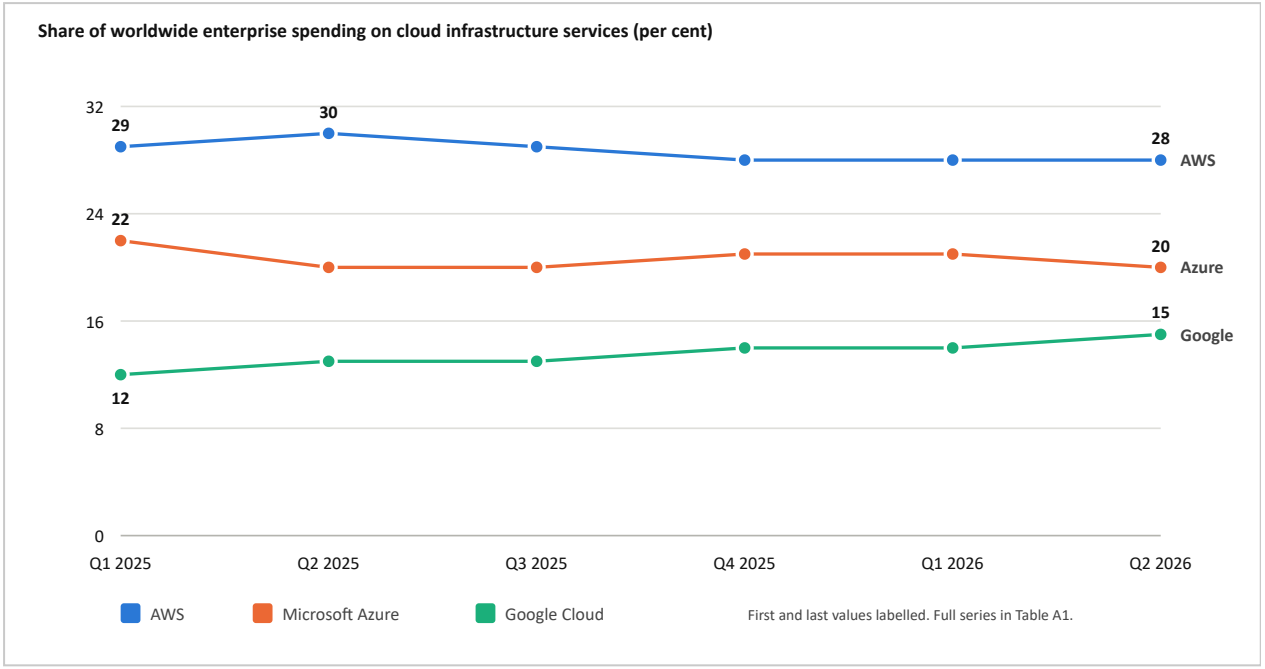


Figure 2. Worldwide cloud infrastructure market share, first quarter of 2025 to second quarter of 2026. Data from Synergy Research Group quarterly releases [1]–[6].

3.3 Market size

A share figure carries no information about volume, so Figure 3 plots the total those percentages are taken from. Quarterly spending rose from 94.0 billion dollars to 143.4 billion dollars across the same six quarters, an increase of 52.6 per cent. The consequence for the reading of Figure 2 is direct: AWS lost two percentage points of share while its absolute revenue grew substantially, because the market itself grew by half over the period.

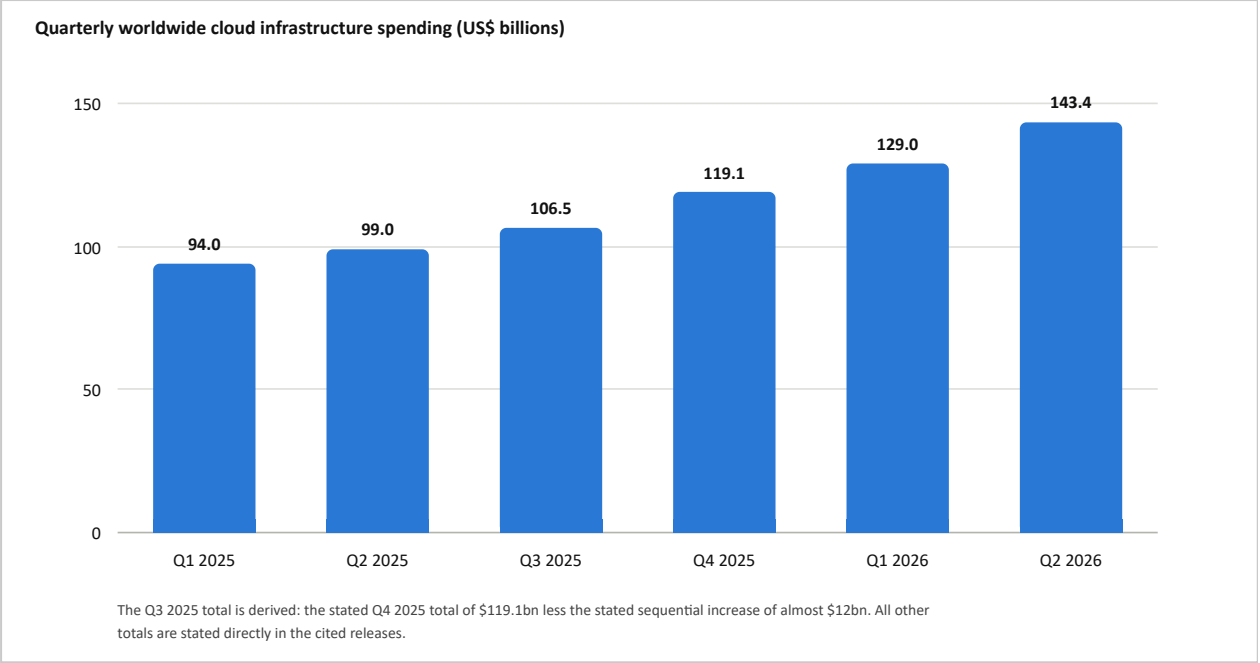


Figure 3. Quarterly worldwide cloud infrastructure spending, first quarter of 2025 to second quarter of 2026. Data from Synergy Research Group quarterly releases [1]–[6]; the third quarter of 2025 is derived as described in Section 3.1.

3.4 Selection of a provider for this study

AWS was selected for three reasons. It is the largest of the three by share, so more workloads run on it than on either competitor and its behaviour is the most consequential to understand. It is also the oldest: EC2 entered public beta in 2006, and the abstractions it introduced, among them the machine image, the security group and the availability zone, were subsequently adopted in near identical form by both competitors, so the material transfers (Section 11 sets out the correspondence). Finally, its price list is published in machine readable form [7], which allows the cost analysis in Section 8 to be reproduced by a reader rather than taken on trust.

The selection is not an endorsement. Section 11 compares the three providers directly and Section 12 sets out the costs and risks that attach to the choice.

4 AWS global infrastructure

4.1 Regions, Availability Zones and edge locations

Three nested concepts carry most of the availability design in AWS, and every later decision inherits from them [11].

- A *Region* is a separate geographic area, identified by a code such as `us-east-1` or `eu-west-1`, with its own price list, its own service catalogue and its own failure domain. Regions are isolated from one another by design, and data does not move between them unless the customer moves it.
- An *Availability Zone* is one or more discrete data centres within a region, with independent power, cooling and network feeds, connected to sibling zones by low latency private links. The zone is the unit of blast radius.

- An *edge location* is a point of presence for the content delivery and DNS services. Edge locations are far more numerous than regions and hold cached content close to users.

The practical rule that follows is worth stating explicitly, because it governs the architecture in Section 7. A workload confined to a single Availability Zone has the availability of a single data centre. Distributing the same workload across two zones within one region removes an entire class of outage and costs nothing additional in instance price, since instances are billed individually. Figures 4 and 7 both show two zones for this reason.

4.2 Network structure inside a region

Figure 4 shows the containment relationship. A Virtual Private Cloud (VPC) is a software defined network that spans every Availability Zone in its region. A subnet, by contrast, exists in exactly one zone. That asymmetry is what makes multi zone design a subnet level decision rather than an instance level one.

The distinction between the public and private subnets in the figure is a routing distinction, not a firewall one. The private subnet has no route to the internet gateway, so the database placed in it is unreachable from outside the VPC regardless of any security group rule. Both mechanisms are discussed in Section 10.3.

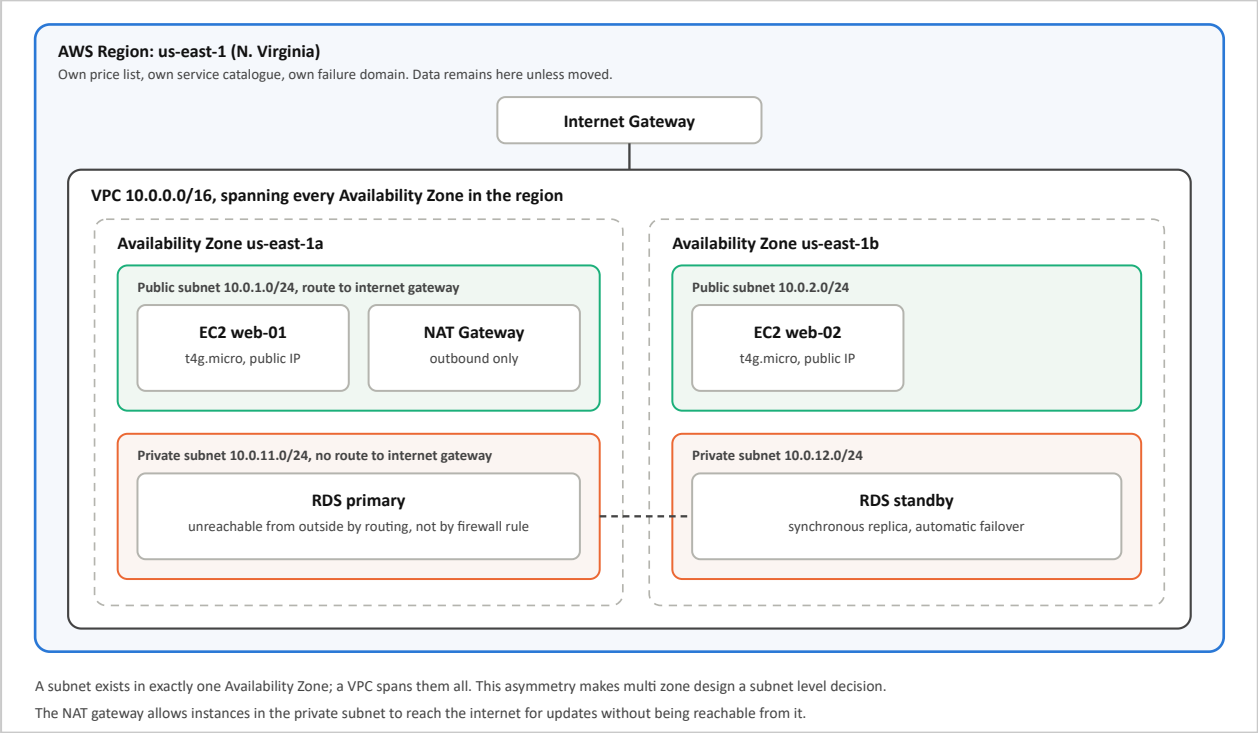


Figure 4. Region, Availability Zone, VPC and subnet structure for a two zone deployment. Boxes denote containment rather than traffic; traffic is shown in Figure 7. The dashed connector between the two database instances is the synchronous replication link.

5 Anatomy of an EC2 virtual machine

5.1 Components of an instance

A request to launch an instance is assembled from eight largely independent choices. Most of the configuration errors recorded in Section 6.8 originate in one of them, so it is useful to separate the

instance into its parts before building one. Figure 5 shows the eight components and the constraints that attach to each.

Three of the choices are fixed for the life of the instance: the machine image, the placement (the subnet, and therefore the Availability Zone) and the name of the key pair. The remainder can be changed later, the instance type only while the instance is stopped, and the security group and the identity role while it runs. Distinguishing the two categories is the difference between modifying an instance and rebuilding it.

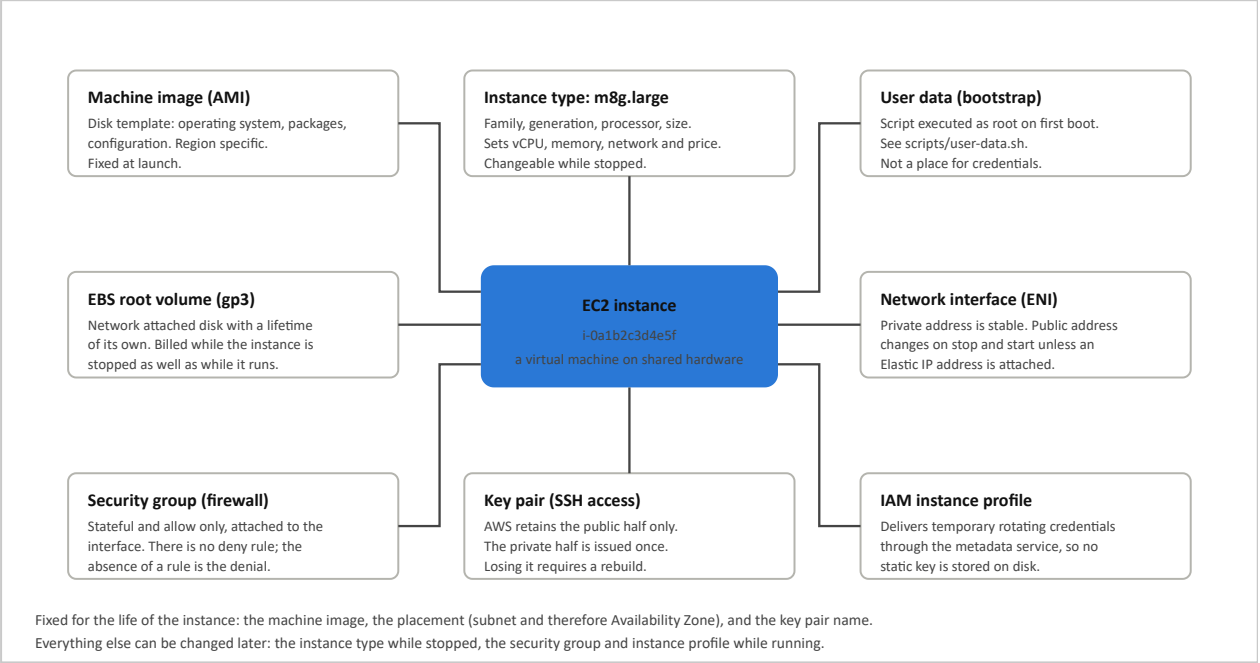


Figure 5. Components of a single EC2 instance and the constraint attached to each. Behaviour as documented in the EC2 User Guide [11].

5.2 Reading an instance type name

The instance type is a structured identifier rather than an arbitrary label. Taking `m8g.large` and reading left to right: `m` is the family (general purpose), `8` is the generation, `g` is the processor (AWS Graviton, where `i` would indicate Intel and `a` AMD), and `large` is the size. Sizes double as they ascend, so `large` provides 2 vCPU, `xlarge` 4 and `2xlarge` 8.

Price follows that progression almost exactly. In Table 6 the `m8g` family is priced at 0.0898, 0.1795 and 0.3590 dollars per hour for those three sizes, which is a doubling to within a hundredth of a cent at each step. Two `m8g.large` instances therefore cost the same as one `m8g.xlarge` while occupying two Availability Zones instead of one. This is the arithmetic behind the preference for horizontal over vertical scaling; the two are equivalent in price, and differ in availability.

5.3 Instance families

Table 2 lists the six families, the workload characteristic each is optimised for, and a representative price. The range between the smallest and largest entries is wide enough to require comment. A `p5.48xlarge` instance at 55.04 dollars per hour costs approximately 40,179 dollars per month if left running continuously, which is about 6,550 times the cost of the `t4g.micro` instance used for the practical work in Section 6. Cost governance is discussed in Section 8.4.

Table 2. EC2 instance families, the characteristic each is optimised for, and a representative On-Demand price. Prices for US East (N. Virginia) on Linux, retrieved 7 September 2026 [7].

Family	Letters	Optimised for	Example	US$/hour
General purpose	`m`	Balanced processor to memory ratio. The default selection where the workload profile is not yet known.	`m8g.large`	0.0898
Burstable	`t`	Low baseline processor allocation with credit funded bursts. The cheapest way to run a small or largely idle service.	`t4g.micro`	0.0084
Compute optimised	`c`	High processor allocation relative to memory: encoding, build systems, application tiers.	`c8g.large`	0.0798
Memory optimised	`r`, `x`	Large in memory working sets: databases, caches, analytical processing.	`r8g.large`	0.1178
Storage optimised	`i`, `d`	Locally attached NVMe storage for high random input and output rates.	`i4i.large`	0.1720
Accelerated computing	`g`, `p`	Graphics processors for model training, inference and rendering.	`p5.48xlarge`	55.0400

6 Provisioning a virtual machine

This section provisions the same virtual machine three times: through the web console, through the command line interface, and as declarative infrastructure code. The specification is held constant across all three so that the comparison in Section 6.5 isolates the method rather than the result.

6.1 Target configuration

Table 3. Target configuration built by all three provisioning methods, with the reason for each selection.

Parameter	Value	Reason
Region	`us-east-1`	Lowest prices for most instance types, and the region from which every price in this report is quoted.
Machine image	Amazon Linux 2023, arm64	Resolved at run time from the Systems Manager parameter store rather than hard coded, since image identifiers are region specific and change with each security update.
Instance type	`t4g.micro`	0.0084 dollars per hour, approximately 6.13 dollars per month. The least expensive type capable of running the service.
Root volume	20 GiB gp3, encrypted, deleted on termination	gp3 is faster and cheaper than gp2 at equal capacity. Deletion on termination prevents an orphaned volume from continuing to bill.
Security group	TCP 22 from one address; TCP 80 from any address	The web service is public by intent. The administrative channel is not.
Key pair	ed25519, private half retained locally	AWS stores the public half only, so the private key never leaves the operator's machine.
Metadata service	IMDSv2 required	Closes the request forgery path to credential disclosure described in Section 10.6.
Bootstrap	`scripts/user-data.sh`	Installs and starts the web server, and publishes a page identifying which instance answered the request.

6.2 Method A: the management console

The console is the appropriate method for a first build because it names every decision explicitly. It is a poor method for a repeated build, because none of the selections are recorded anywhere afterwards. The procedure is as follows.

1. Select the region first, from the control at the top right. Every subsequent screen is scoped to it, and an instance launched in the wrong region is not visible from the correct one.
2. Open EC2, then Instances, then Launch an instance.
3. Set the name and tags. A name tag of `cloud-project-web` is used here. Tags are the mechanism by which resources are later located, grouped and attributed to a cost centre.
4. Select the machine image. Choose Amazon Linux 2023, then set the architecture selector to 64-bit Arm. This must agree with the processor implied by the instance type chosen next; an x86 image cannot be launched on a Graviton instance type.
5. Select the instance type `t4g.micro`. The screen states the vCPU count, memory and On-Demand price, which can be checked against Table 2.
6. Create a key pair of type ed25519 in `.pem` format. The browser downloads the private key once. There is no second copy and no recovery procedure.
7. Edit the network settings. Select a specific subnet and note which Availability Zone it belongs to, since this determines where the instance physically runs. Enable the automatic assignment of a public address. Create a security group with two rules: SSH on port 22 with the source set to My IP,

and HTTP on port 80 with the source set to anywhere. The console offers Anywhere as a single click option for SSH as well; Section 6.8 explains why it should not be taken.

8. Configure storage as 20 GiB of type `gp3`, and enable encryption under the advanced options.
9. Under advanced details, set the metadata version to V2 only (token required) and paste the contents of `scripts/user-data.sh` into the user data field.
10. Launch the instance. The state proceeds from *pending* to *running*, and the two status checks proceed from initialising to 2/2 passed. The instance is reachable at 2/2, not at *running*.

6.3 Method B: the command line interface

The complete script, including the removal path, is `scripts/create-ec2-cli.sh`. The launch call itself is reproduced below.

```
aws ec2 run-instances \
  --region us-east-1 \
  --image-id "$AMI_ID" \
  --instance-type t4g.micro \
  --key-name cloud-project-cli-key \
  --security-group-ids "$SG_ID" \
  --user-data file://scripts/user-data.sh \
  --metadata-options "HttpTokens=required,HttpEndpoint=enabled,HttpPutResponseHopLimit=1" \
  --block-device-mappings '[{"DeviceName":"/dev/xvda","Ebs":{
        "VolumeSize":20,"VolumeType":"gp3","Encrypted":true,"DeleteOnTermination":true}}]' \
  --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=cloud-project-cli}]"
```

Two details in the surrounding script matter more than the launch call. The machine image is resolved at run time from the Systems Manager public parameter store, so the script continues to work after Amazon publishes a patched image:

```
AMI_ID=$(aws ssm get-parameters --region us-east-1 \
  --names /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-arm64 \
  --query 'Parameters[0].Value' --output text)
```

and the administrative rule in the security group is computed from the operator's own address rather than typed:

```
MY_IP="$(curl -s https://checkip.amazonaws.com)/32"
aws ec2 authorize-security-group-ingress --group-id "$SG_ID" \
  --ip-permissions "IpProtocol=tcp,FromPort=22,ToPort=22,IpRanges=[{CidrIp=$MY_IP}]"
```

> **Note.** The script creates resources but does not record them. Executing it twice produces two instances, two security groups and two charges, and there is no reliable way to modify or remove what it created except by searching for tags afterwards. That limitation is the argument for the third method.

6.4 Method C: infrastructure as code

The configuration is `terraform/main.tf`, with inputs in `variables.tf` and results in `outputs.tf` [12]. The instance resource is reproduced below.

```
resource "aws_instance" "vm" {
  ami                         = data.aws_ami.al2023_arm.id
  instance_type               = var.instance_type
  subnet_id                   = data.aws_subnets.default.ids[0]
  vpc_security_group_ids      = [aws_security_group.web.id]
  key_name                    = aws_key_pair.this.key_name
  associate_public_ip_address = true
  user_data                   = file("${path.module}/../scripts/user-data.sh")

  root_block_device {
    volume_size           = 20
    volume_type           = "gp3"
    encrypted             = true
    delete_on_termination = true
  }

  metadata_options {
    http_tokens = "required"     # IMDSv2 required
  }

  tags = { Name = var.name, Project = "cloud-computing-report" }
}
```

The build and removal sequence is:

```
ssh-keygen -t ed25519 -f ~/.ssh/cloud-project -C cloud-project
chmod 400 ~/.ssh/cloud-project

cd terraform
terraform init
terraform plan  -var="my_ip_cidr=$(curl -s https://checkip.amazonaws.com)/32"
terraform apply -var="my_ip_cidr=$(curl -s https://checkip.amazonaws.com)/32"
terraform destroy -var="my_ip_cidr=$(curl -s https://checkip.amazonaws.com)/32"
```

Three properties follow from this method that neither of the others provides. The `plan` operation reports what will change before anything changes. The state file makes the deployed infrastructure comparable against the configuration, so it can be reviewed in the same way as source code. The `destroy` operation removes what was created and nothing else, which in a student account is the difference between an exercise costing a few dollars and one costing considerably more.

The configuration also refuses one specific misuse. A validation block in `variables.tf` rejects a value of `0.0.0.0/0` for the administrative source address, so the most common security error in this exercise fails at plan time rather than succeeding silently.

6.5 Comparison of the three methods

Table 4. Comparison of the three provisioning methods against the single specification in Table 3.

Criterion	Console	Command line	Terraform
Elapsed time to first instance	about 3 minutes	about 90 seconds	about 2 minutes after initialisation
Reproducible identically	No	Yes, but duplicates on repetition	Yes; repetition converges rather than duplicating
Reviewable before application	No	No	Yes, through `terraform plan`
Complete removal	Manual, with a risk of orphaned resources	Scripted, by tag	`terraform destroy`
Appropriate use	Learning and one off inspection	Automation within other scripts and pipelines	Anything required to exist tomorrow

6.6 Connection and verification

```
chmod 400 ~/.ssh/cloud-project          # ssh refuses a world readable key
ssh -i ~/.ssh/cloud-project ec2-user@<public-ip>

# on the instance, confirm the bootstrap script completed
sudo systemctl status nginx
sudo cat /var/log/cloud-init-output.log

# confirm IMDSv2 is enforced: the version 1 style request must fail
curl -s http://169.254.169.254/latest/meta-data/instance-id     # 401 Unauthorized
TOKEN=$(curl -sX PUT http://169.254.169.254/latest/api/token \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 60")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/instance-id           # i-0a1b2c...
```

Requesting `http://<public-ip>/` in a browser returns the page written by `scripts/user-data.sh`, which reports the instance identifier, the instance type and the Availability Zone in which it is running. Those three values allow the same page to serve as a load balancer demonstration later, since the responding instance can be identified from the response.

6.7 Instance lifecycle and billing states

Figure 6 gives the instance lifecycle as a state machine, annotated with what each state bills. Instance hours accrue only in the *running* state. The attached storage volume, however, is billed per gigabyte month for as long as the volume exists, which includes the whole of the *stopped* state. Stopping an instance therefore reduces the charge without ending it. Only termination ends the charge, and only if the volume was configured to be deleted with the instance.

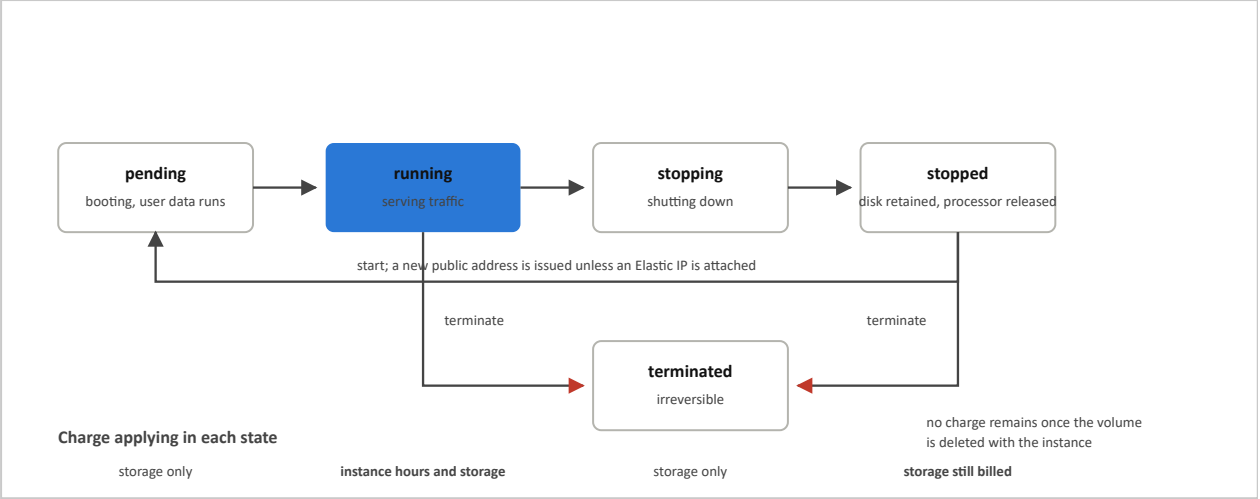


Figure 6. Instance lifecycle with the charge applying in each state. Lifecycle states as documented in [11].

6.8 Configuration errors observed in practice

Four errors account for most of the marks lost on this exercise and most of the incidents in small accounts.

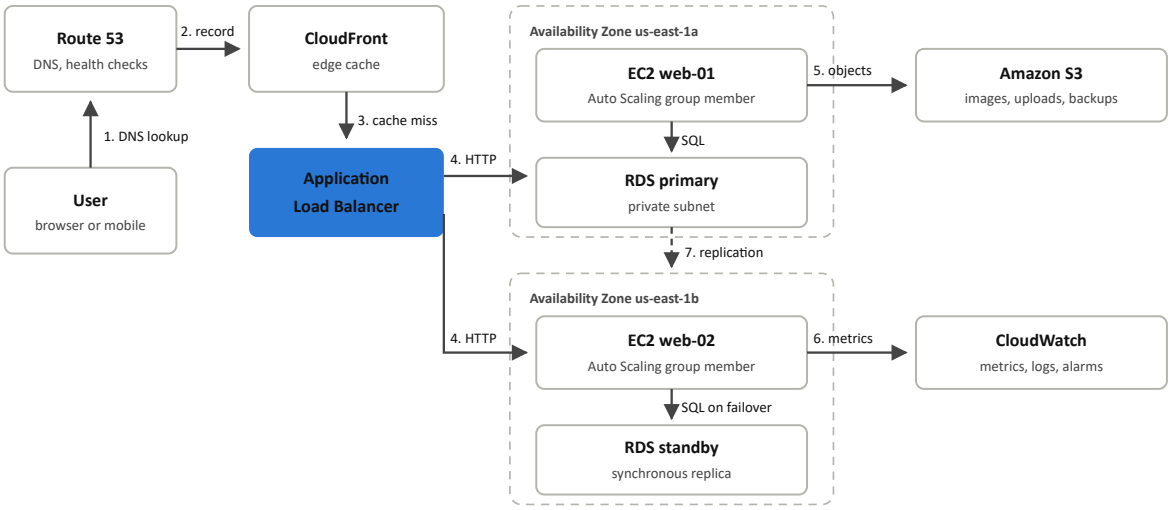
1. *Administrative access opened to the whole internet.* A newly exposed SSH port is located by automated scanners within minutes. Port 22 should be scoped to a single address, or opened not at all if Systems Manager Session Manager is used instead.
2. *Mishandling of the private key.* The key is issued once, must be readable only by its owner, and must not be committed to a repository. There is no recovery path if it is lost.
3. *Version 1 of the metadata service left enabled.* The consequences are set out in Section 10.6.
4. *Stopping an instance instead of terminating it.* Storage continues to accrue charges in the stopped state, as Figure 6 shows.

7 Reference architecture and data flow

A single instance is a demonstration rather than an architecture. Figure 7 shows the same application after it has been made able to survive the loss of a data centre and to absorb a change in load. Where Figure 4 showed containment, the arrows in Figure 7 carry traffic, and each is labelled with what moves along it.

The request path proceeds as follows. The client resolves the service name at Route 53, whose health checks allow DNS itself to direct traffic away from a failed region. The resolved name points at CloudFront, which serves cached static content from an edge location; requests answered there never reach the instances at all, which is the least expensive way to serve them. Requests that miss the cache pass to the Application Load Balancer, which distributes them across the instances in both Availability Zones and withdraws any instance that fails its health check without operator intervention. The instances read and write persistent objects in S3 and relational data in RDS, which maintains a synchronous replica in the second zone. Metrics and logs are collected by CloudWatch, whose alarms are what trigger the automatic scaling of the instance group.

The organising principle is that state is pushed outward, into S3 and RDS, so that the compute layer holds none of it. An instance that holds no state can be replaced without consequence, and replacement without consequence is the precondition for automatic scaling and for automatic recovery. Nothing in this architecture alters the instance itself; availability is a property of what surrounds it.



Function of each hop

1 and 2. Route 53 health checks allow DNS to direct traffic away from a failed region. 3. CloudFront absorbs static traffic at the edge, so those requests never reach EC2. 4. The load balancer withdraws an unhealthy instance from rotation without operator action. 5. Persistent state is held in S3 rather than on the instance. 6. CloudWatch alarms trigger scaling. 7. Synchronous replication to a second zone allows the database to survive the loss of a data centre.

State is pushed outward into S3 and RDS so that the compute layer becomes replaceable, which is the precondition for automatic scaling.

Figure 7. Data flow for a two zone web application. Arrows carry traffic and are labelled with the content of each exchange; compare with the containment relationships in Figure 4.

8 Cost analysis

8.1 Purchase models

EC2 sells an identical virtual machine under four commercial models. The instance does not differ between them; what differs is the commitment the customer makes and therefore the price paid. Figure 8 expresses each model as the proportion of the On-Demand list price payable under it, and Table 5 states the same figures with their terms.

The published reductions are maximum values rather than measured averages. Actual reductions vary with instance type, region and term length, and interruptible capacity carries the additional condition that it may be reclaimed at two minutes' notice.

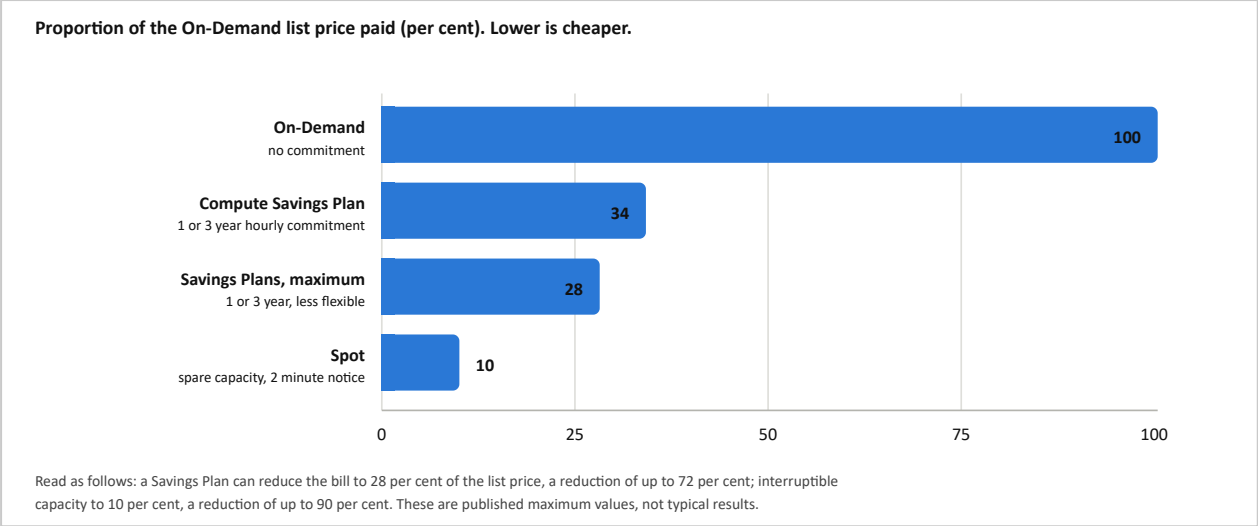


Figure 8. Proportion of the On-Demand price payable under each purchase model. Published maximum reductions from the AWS EC2 pricing page [8] and the Compute Savings Plans page [9], accessed 7 September 2026.

Table 5. EC2 purchase models with published maximum reductions and the commitment required. Source: [8], [9].

Model	Maximum reduction	Proportion of list paid	Commitment
On-Demand	—	100 %	None
Compute Savings Plan	66 %	34 %	One or three year commitment to an hourly spend, flexible across family and region
Savings Plans, maximum	72 %	28 %	One or three year commitment to an hourly spend, less flexible
Spot	90 %	10 %	None; capacity may be reclaimed at two minutes' notice

8.2 Processor selection

AWS designs its own Arm based server processors under the Graviton name and prices the resulting instances below their Intel equivalents at equal processor and memory allocations. Figure 9 compares three matched pairs from the current generation. Because the specification on either side of each pair is identical, the difference is attributable to price alone.

The three pairs give reductions of 10.9, 10.5 and 11.0 per cent. The consistency across families suggests a deliberate pricing position rather than incidental variation. The constraint attached to it is real but narrow: the software must have an arm64 build available. For interpreted languages and for containers based on current images this condition is generally satisfied.

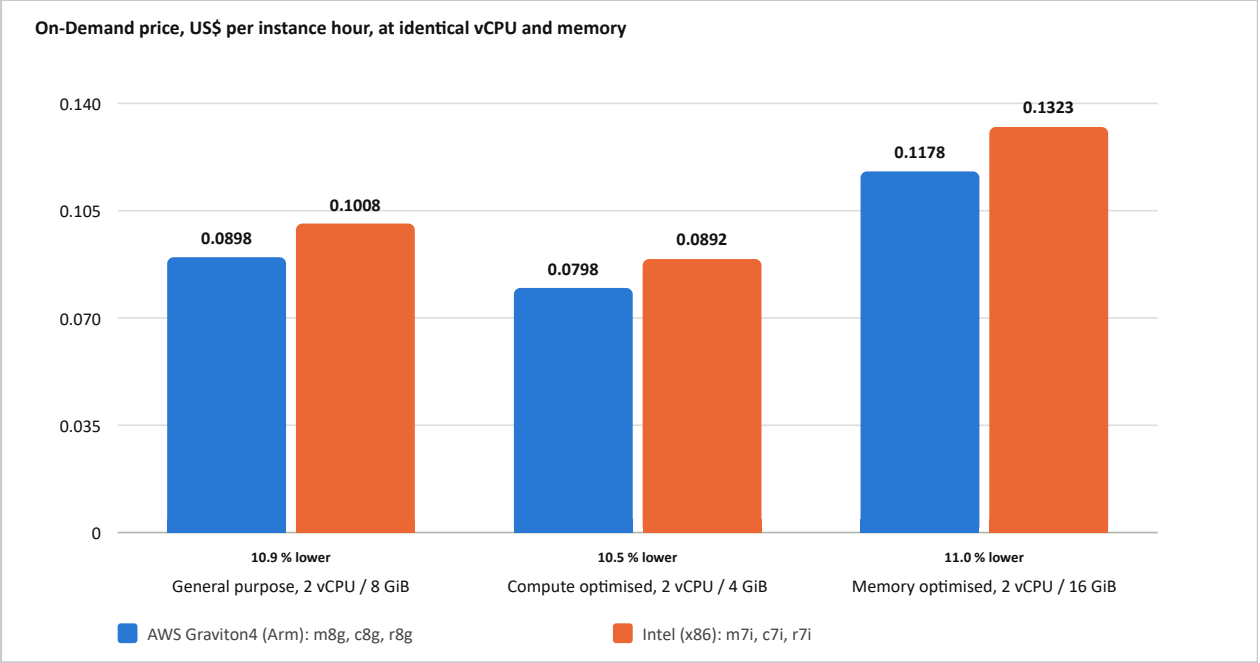


Figure 9. Graviton4 and Intel On-Demand prices at matched processor and memory allocations. Prices for US East (N. Virginia) on Linux, retrieved 7 September 2026 [7]; reductions derived as stated in Table A2.

Table 6. On-Demand list prices for the instance types referenced in this report. US East (N. Virginia), Linux, retrieved 7 September 2026 [7]. Monthly figures are derived at 730 hours. Storage, data transfer and address charges are additional.

Instance type	Processor	vCPU	Memory	US$/hour	US$/month
`t4g.micro`	Graviton2 (Arm)	2	1 GiB	0.0084	6.13
`t3.micro`	Intel (x86)	2	1 GiB	0.0104	7.59
`t4g.small`	Graviton2 (Arm)	2	2 GiB	0.0168	12.26
`t3.small`	Intel (x86)	2	2 GiB	0.0208	15.18
`c8g.large`	Graviton4 (Arm)	2	4 GiB	0.0798	58.22
`c7i.large`	Intel (x86)	2	4 GiB	0.0892	65.12
`m8g.large`	Graviton4 (Arm)	2	8 GiB	0.0898	65.52
`m7i.large`	Intel (x86)	2	8 GiB	0.1008	73.58
`r8g.large`	Graviton4 (Arm)	2	16 GiB	0.1178	86.02
`r7i.large`	Intel (x86)	2	16 GiB	0.1323	96.58
`c8g.xlarge`	Graviton4 (Arm)	4	8 GiB	0.1595	116.44
`i4i.large`	Intel with local NVMe	2	16 GiB	0.1720	125.56
`m8g.xlarge`	Graviton4 (Arm)	4	16 GiB	0.1795	131.04
`x8g.large`	Graviton4 (Arm)	2	32 GiB	0.1954	142.64
`c8g.2xlarge`	Graviton4 (Arm)	8	16 GiB	0.3190	232.87
`m8g.2xlarge`	Graviton4 (Arm)	8	32 GiB	0.3590	262.07
`g6.xlarge`	AMD with NVIDIA L4	4	16 GiB	0.8048	587.50
`p5.48xlarge`	AMD with 8 × NVIDIA H100	192	2048 GiB	55.0400	40,179.20

8.3 Charges outside the instance hour

The instance hour is the figure most often optimised and the one least likely to produce a surprise. Three other charges are more common causes of an unexpected invoice.

- *Block storage.* Volumes are billed per gigabyte month for as long as they exist, whether the instance runs, is stopped, or was terminated while leaving the volume behind. The current rate is published on the AWS storage pricing page. It is not quoted here because that page serves its prices from a client side template rather than from the machine readable feed used elsewhere in this report, and every figure quoted here was taken from a source that could be read directly.
- *Data transfer.* Inbound transfer is not charged. Outbound transfer to the internet is charged, and traffic between Availability Zones is charged in both directions. An architecture that exchanges large volumes between zones pays for every exchange.
- *Idle public addresses.* Public IPv4 addresses are charged hourly, including allocated addresses that are not attached to a running instance.

8.4 Order of cost control measures

The measures below are ordered by effect per unit of effort. Applying them in this order produces most of the available saving before any engineering work is required.

1. Stop what is not in use. A development instance running only during office hours costs approximately a quarter of one running continuously. No other measure approaches this.
2. Correct the size. Instances sustaining low processor utilisation should be moved down the size ladder, using the utilisation metrics already collected by CloudWatch.
3. Commit the steady baseline. Once demand is genuinely stable, a Savings Plan removes up to 72 per cent of the cost of that baseline [8].
4. Use interruptible capacity for interruptible work. Batch processing, build agents and checkpointed training tolerate a two minute eviction notice and should not be paying the list price.
5. Select the processor last. The 11 per cent available from Figure 9 is worth taking, but it is the smallest of the five.

9 Use cases

Each case below identifies the workload, the instance family suited to it and the purchase model that applies, since as Section 8.1 established the commercial choice moves the total by more than the hardware choice does.

9.1 Public web and application hosting

The case built in Section 6 and extended in Section 7. An Auto Scaling group of general purpose instances sits behind an application load balancer across two zones, with state held in RDS and S3 so that instances remain replaceable. The `m8g` and `c8g` families suit the application tier, with `t4g` for small or intermittently used sites. A Savings Plan covers the steady baseline and On-Demand capacity covers the peak. The economic argument rests on the ratio of peak to trough demand: an organisation sizing owned hardware for its busiest trading day pays for that capacity for the remaining 364 days of the year.

9.2 Batch processing and scientific computing

Video transcoding, sequence alignment, Monte Carlo simulation and overnight data processing share two properties. The work parallelises readily and it tolerates interruption, which makes it the natural candidate for the interruptible capacity market. Compute optimised `c8g` instances purchased at spot prices, with checkpointing so that an eviction costs minutes rather than the whole run, is the usual configuration. Five hundred instances for one hour cost the same as one instance for five hundred hours and return the result the same day.

9.3 Machine learning training and inference

Training runs on `p5` instances carrying eight H100 accelerators at 55.04 dollars per hour, while serving runs on smaller `g6` instances. This is the category of workload to which Synergy attributes the recent acceleration in cloud spending, reporting growth of 165 per cent year on year for generative AI services in the second quarter of 2026 [6]. The alternative to renting is a machine costing on the order of 40,000 dollars per month to operate, which converts the decision from a capital question into a utilisation question.

9.4 Development, test and continuous integration

These environments are ephemeral by nature: one preview environment per change, removed when the change is merged. Burstable `t4g` instances suit them, because such machines are idle for most of their existence, which is the condition the credit based burstable model is designed for. On-Demand purchasing with a scheduled shutdown is appropriate, with spot capacity for build agents. Where the environment is defined by the same configuration files as production, as it is in the accompanying Terraform, equivalence between the two is exact rather than approximate.

9.5 Migration and disaster recovery

Two related cases: the relocation of existing servers, and a standby site held in a second region. For a migration the instance should initially match the source machine and be resized only once real utilisation data exists, since migrations that resize on the first day usually resize incorrectly. On-Demand purchasing suits the migration period and a commitment suits the steady state that follows. A standby site costing a few dollars per month while idle compares favourably with a second data centre costing as much as the first whether or not it is ever used.

10 Security

10.1 The shared responsibility model

AWS states the division as security *of* the cloud against security *in* the cloud. The provider is responsible for the hardware, the hypervisor, the physical facilities and the software of its managed services. The customer is responsible for everything above that line. As Figure 1 showed, under infrastructure as a service the line sits high: the guest operating system and its patch level, firewall rules, identity configuration, encryption settings and application code are all customer responsibilities.

10.2 Identity

The account root user should not be used for routine work. Multi factor authentication should be enabled on it and the credentials stored securely and left unused. Applications running on an instance should receive an IAM role attached to the instance rather than a static access key written into an image or a configuration file. The role supplies temporary credentials that rotate automatically, delivered through the instance metadata service, which is the reason Section 10.6 treats that service as a security control.

10.3 Network

Two independent mechanisms apply and both should be used. Security groups are stateful, allow only firewalls attached to the network interface; there is no deny rule, and the absence of an allow rule constitutes the denial. Route tables determine whether a subnet can reach the internet at all. In Figure 4 the database is unreachable because of its route table rather than because of a firewall rule, so a later misconfiguration of the security group does not expose it. Defence in depth here means that the two layers fail independently.

10.4 Data

Encryption of block storage volumes at rest is a single configuration setting with no measurable performance cost, and it is set in the accompanying Terraform configuration. Transport layer security should be terminated at the load balancer using a certificate issued by the provider's certificate manager. Backups should be taken as volume snapshots or through the automated backup facility of the managed database service. A backup that has never been restored is an assumption rather than a backup, so at least one restoration should be performed deliberately.

10.5 Monitoring and audit

CloudTrail records every API call made within the account, which is what allows an unexpected resource to be attributed to an identity and a time. CloudWatch raises alarms on metric thresholds; in a small account a billing alarm is the single most useful alarm to configure. AWS Config can detect a security group that opens an administrative port to the internet at the point the rule is created rather than at the point it is exploited.

10.6 The instance metadata service

The instance metadata service is reachable from the instance at the link local address 169.254.169.254 and returns, among other things, the temporary credentials associated with the instance role. Under version 1 of the service a request was an ordinary HTTP GET. A server side request forgery defect in any application running on the instance could therefore be directed at that address and would return live credentials for the account.

Version 2 requires the caller to obtain a session token by an HTTP PUT before any data is returned, and applies a network hop limit so that the token cannot be retrieved through a proxy or across a container boundary. Enforcement is a single setting, expressed in the accompanying configuration as `http_tokens = "required"`, and it closes the whole class of defect described above. The verification procedure in Section 6.6 confirms that the version 1 style request fails.

11 Comparison of the three major providers

The three providers converged on a common set of abstractions under different names. A reader who understands Figures 4 and 5 already understands the equivalent constructs on the other two platforms; Table 7 gives the correspondence.

Table 7. Equivalent services and concepts across the three major providers. Market shares for the second quarter of 2026 from [6].

Concept	Amazon Web Services	Microsoft Azure	Google Cloud
Virtual machine service	EC2	Virtual Machines	Compute Engine
Machine image	AMI	Managed Image or Gallery	Image
Private network	VPC	Virtual Network (VNet)	VPC network
Instance firewall	Security group	Network security group	VPC firewall rule
Failure domain within a region	Availability Zone	Availability Zone	Zone
Object storage	S3	Blob Storage	Cloud Storage
Managed relational database	RDS	Azure SQL, Flexible Server	Cloud SQL
Function as a service	Lambda	Azure Functions	Cloud Run functions
Committed use discount	Savings Plans, Reserved Instances	Reserved VM Instances, Savings Plan	Committed use discounts
Interruptible capacity	Spot Instances	Spot Virtual Machines	Spot VMs
In house Arm processor	Graviton	Cobalt	Axion
Market share, Q2 2026	28 %	20 %	15 %

The providers differ in emphasis rather than in capability. AWS has the widest service catalogue and the deepest third party ecosystem, which follows from its decade of priority. The advantage of Azure is commercial and organisational: existing enterprise agreements, directory services already deployed, and licensing terms favourable to organisations already running Microsoft server products. Google Cloud competes on data analysis and machine learning, and is taking share fastest of the three, having gained three percentage points over the six quarters plotted in Figure 2. For a workload consisting of a conventional Linux web application, as in Section 6, the three are close enough in capability that the decision is usually settled by existing commitments, by the skills available in the organisation, and by the discount negotiated.

12 Limitations and risks

12.1 Supplier dependence

An EC2 instance running Linux is portable, since it is an ordinary Linux machine. Dependence accumulates in the services around it: identity policies, the storage semantics of S3, the function service, the managed databases, all of the components that make the platform worth using. The cost of leaving therefore rises with the quality of the engineering, which is a property of the commercial model rather than an accident of it.

12.2 Asymmetric transfer pricing

Data entering the platform is not charged; data leaving it is. The asymmetry discourages architectures that span providers and gives data gravity a direction, since it is consistently cheaper to move computation to the data than the reverse.

12.3 Elasticity of cost

The self service property that provisions a server in ninety seconds provisions a cluster of accelerated instances in the same ninety seconds, at a cost several thousand times higher. Without budget alarms, resource tagging and automated configuration rules, an organisation discovers the consequence at the end of the billing period.

12.4 Market concentration

Three suppliers hold approximately two thirds of the market, and 67 per cent of public cloud specifically [6]. A regional failure at any one of them removes a visible fraction of the internet, and no individual customer's architecture alters that systemic exposure.

12.5 Jurisdiction and capacity

The choice of region determines which legal jurisdiction governs the stored data, which for regulated workloads is a question that precedes the technical design rather than following it. Capacity is also not unlimited in practice. Accelerated instance types in particular are subject to real constraints and to per account quotas, and an `InsufficientInstanceCapacity` response is a condition that must be handled rather than an anomaly.

13 Conclusions

Provisioning a virtual machine on AWS is a small technical task. Section 6 completed it by three separate methods, the slowest of which took about three minutes. Because the task itself is trivial, the difficulty moves elsewhere: to the administrative port left open to the internet, the metadata service left at version 1, the instance stopped rather than terminated, and the list price paid for a workload whose demand has been stable for two years. Each of those is a decision taken in seconds and paid for over months.

Three findings emerge from the analysis. First, the commercial purchase model changes the cost of a workload by up to 72 per cent while the choice of processor changes it by about 11 per cent, so the sequence in Section 8.4 places the commercial decisions first. Second, availability is a property of the architecture surrounding an instance rather than of the instance itself; nothing in the reference architecture of Section 7 modifies the virtual machine. Third, the difference between the three provisioning methods is not in the time they take but in whether the result can be reviewed before it is applied and removed reliably afterwards.

The practical form of the exercise is therefore wider than the instruction to launch an instance. The instance should be expressed as configuration, its access scoped to what the workload requires, its state and the charge associated with that state understood, and the resources removed when the demonstration ends. The market these skills apply to grew 43 per cent in the year to the second quarter of 2026 and has roughly doubled in eleven quarters [6]. Whatever share of that growth is ultimately

attributable to demand for artificial intelligence services, the operational disciplines examined in this report sit underneath all of it.

References

[1] Synergy Research Group, "AI helps cloud market growth rate jump back to almost 25% in Q1," press release, 1 May 2025. srgresearch.com/articles/ai-helps-cloud-market-growth-rate-jump-back-to-almost-25-in-q1 (accessed 7 Sep. 2026).

[2] Synergy Research Group, "Q2 cloud market nears $100 billion milestone, and it is still growing by 25% year over year," press release, 31 Jul. 2025. srgresearch.com/articles/q2-cloud-market-nears-100-billion-milestone-and-its-still-growing-by-25-year-over-year (accessed 7 Sep. 2026).

[3] Synergy Research Group, "Cloud market growth rate rises again in Q3; biggest ever sequential increase," press release, 31 Oct. 2025. srgresearch.com/articles/cloud-market-growth-rate-rises-again-in-q3-biggest-ever-sequential-increase (accessed 7 Sep. 2026).

[4] Synergy Research Group, "GenAI helps drive quarterly cloud revenues to $119 billion as growth rate jumped yet again in Q4," press release, 5 Feb. 2026. srgresearch.com/articles/genai-helps-drive-quarterly-cloud-revenues-to-119-billion-as-growth-rate-jumped-yet-again-in-q4 (accessed 7 Sep. 2026).

[5] Synergy Research Group, "Cloud market annual revenue run rate topped half a trillion dollars in Q1 as growth surge continues," press release, 29 Apr. 2026. srgresearch.com/articles/cloud-market-annual-revenue-run-rate-topped-half-a-trillion-dollars-in-q1-as-growth-surge-continues (accessed 7 Sep. 2026).

[6] Synergy Research Group, "Q2 cloud market passes $143 billion; highest growth rate in eight years," press release, 30 Jul. 2026. srgresearch.com/articles/q2-cloud-market-passes-143-billion-highest-growth-rate-in-eight-years (accessed 7 Sep. 2026).

[7] Amazon Web Services, "Amazon EC2 On-Demand pricing feed, US East (N. Virginia), Linux," machine readable price list queried by the public EC2 pricing page. b0.p.awsstatic.com/pricing/2.0/meteredUnitMaps/ec2/USD/current/ec2-ondemand-without-sec-sel/US East (N. Virginia)/Linux/index.json (retrieved 7 Sep. 2026).

[8] Amazon Web Services, "Amazon EC2 pricing." aws.amazon.com/ec2/pricing/ (accessed 7 Sep. 2026).

[9] Amazon Web Services, "Compute Savings Plans pricing." aws.amazon.com/savingsplans/compute-pricing/ (accessed 7 Sep. 2026).

[10] P. Mell and T. Grance, *The NIST Definition of Cloud Computing*, NIST Special Publication 800-145. Gaithersburg, MD: National Institute of Standards and Technology, Sep. 2011.

[11] Amazon Web Services, *Amazon EC2 User Guide*: instance lifecycle, instance metadata and user data, security groups, and Amazon EBS volume types. docs.aws.amazon.com/ec2/ (accessed 7 Sep. 2026).

[12] HashiCorp, "Terraform AWS provider: `aws_instance` resource." registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/instance (accessed 7 Sep. 2026).

> **Note on sources.** Market share figures were taken from the Synergy Research Group releases themselves rather than from secondary articles reporting them, and prices were read from the machine readable pricing feed rather than from a pricing article, because both categories of figure are commonly misquoted after one or two rehandlings. The dataset behind every figure in this report, including the exact address and retrieval date of each value, accompanies the report as `data/figures.json`.

Appendix A Figure data

Table A1. Quarterly market data underlying Figures 2 and 3. Source: Synergy Research Group quarterly releases [1]–[6].

Quarter	Market, US$ bn	AWS	Azure	Google Cloud	Release date	Ref.
Q1 2025	94.0	29 %	22 %	12 %	1 May 2025	[1]
Q2 2025	99.0	30 %	20 %	13 %	31 Jul. 2025	[2]
Q3 2025	106.5 (derived)	29 %	20 %	13 %	31 Oct. 2025	[3]
Q4 2025	119.1	28 %	21 %	14 %	5 Feb. 2026	[4]
Q1 2026	129.0	28 %	21 %	14 %	29 Apr. 2026	[5]
Q2 2026	143.4	28 %	20 %	15 %	30 Jul. 2026	[6]

The market total for the third quarter of 2025 is the single derived value in this report. Synergy stated the fourth quarter of 2025 at 119.1 billion dollars and described that quarter as an increase of almost 12 billion dollars over the preceding quarter, giving approximately 106.5 billion dollars [3], [4]. Every other total, and every share percentage, is stated directly in the cited release.

Table A2. Matched processor pairs underlying Figure 9. Prices for US East (N. Virginia) on Linux, retrieved 7 September 2026 [7]. The reduction is derived as (x86 price less Arm price) divided by x86 price.

Pair	Arm	US$/hour	x86	US$/hour	Reduction
General purpose, 2 vCPU / 8 GiB	`m8g.large`	0.0898	`m7i.large`	0.1008	10.9 %
Compute optimised, 2 vCPU / 4 GiB	`c8g.large`	0.0798	`c7i.large`	0.0892	10.5 %
Memory optimised, 2 vCPU / 16 GiB	`r8g.large`	0.1178	`r7i.large`	0.1323	11.0 %
Burstable, 2 vCPU / 1 GiB	`t4g.micro`	0.0084	`t3.micro`	0.0104	19.2 %

The burstable pair is listed for completeness but excluded from Figure 9. Burstable instances price a processor credit mechanism rather than sustained performance, so the comparison is not made on equal terms with the other three pairs.

Appendix B Accompanying files

The files below accompany this report. The configuration in `terraform/` and `scripts/` builds the instance specified in Table 3 and removes it again.

Table B1. Files accompanying this report.

Path	Contents
`report.html`	The source document from which this report was typeset.
`slides.html`	An accompanying presentation covering the same material.
`data/figures.json`	Every numerical value used in this report, each with its source address and retrieval date.
`terraform/main.tf`	The virtual machine expressed as infrastructure code (Section 6.4), with inputs in `variables.tf` and results in `outputs.tf`.
`scripts/create-ec2-cli.sh`	The same instance provisioned through the command line interface (Section 6.3), including the removal path.
`scripts/user-data.sh`	The bootstrap script executed on first boot by both provisioning methods.
`scripts/build-pdf.mjs`	The script that produced this PDF from the source document.
`README.md`	Build and execution instructions.

> **Cost warning.** Running the accompanying configuration incurs real charges, approximately 0.0084 dollars per hour for the instance together with a storage charge for its volume. The removal command in Section 6.4 should be executed once the demonstration is complete. Stopping the instance reduces the charge but does not end it, as Figure 6 shows.